

Deep Learning Practical 5

GANs: from digits to horses

1 Overview

In this practical, we will build a generative adversarial network (GAN), beginning with handwritten digits and then generating colour photographs of horses. We will:

- Implement a generator, discriminator and their alternating optimisation steps.
- Inspect MNIST samples, losses and latent-space interpolations.
- Improve the CIFAR-10 horse model and compare image distributions using KID.

Practicals 5–9 each have a 1 credit summative ‘submission’ component at the bottom. Here, MNIST is used as a quick preparatory exercise, where the CIFAR-10 horse challenge is assessed. The file `d1-practical5.py` [↗](#) supplies the baseline, training loop, scoring and ZIP creation.

```
import torch
from torch import nn
from torch.nn import functional as F
```

The generator maps 16 independent standard Gaussian values to a $1 \times 28 \times 28$ image in $[-1, 1]$. The discriminator then maps an image to a real/fake logit, which the loss uses directly without an output sigmoid.

2 Generator and discriminator

```
class Generator(nn.Module):
    def __init__(self, latent=16, width=16):
        super().__init__()
        self.latent = latent
        self.project = nn.Linear(latent, width*2*7*7)
        self.net = nn.Sequential(
            nn.ConvTranspose2d(width*2, width, 4, 2, 1, bias=False),
            nn.BatchNorm2d(width), nn.ReLU(),
            nn.ConvTranspose2d(width, 1, 4, 2, 1), nn.Tanh())

    def forward(self, z):
        return self.net(self.project(z).reshape(z.size(0), -1, 7, 7))

class Discriminator(nn.Module):
    def __init__(self, width=16):
        super().__init__()
        self.net = nn.Sequential(nn.Conv2d(1, width, 4, 2, 1), nn.LeakyReLU(.2),
                                nn.Conv2d(width*2, 4, 2, 1), nn.LeakyReLU(.2),
                                nn.Flatten(), nn.Linear(width*2*7*7, 1))

    def forward(self, x):
        return self.net(x).squeeze(-1) # logits; no sigmoid before BCEWithLogits

class Model(nn.Module):
    def __init__(self, generator_width=16, discriminator_width=16, latent=16):
        super().__init__()
        self.generator = Generator(latent=latent, width=generator_width)
        self.discriminator = Discriminator(width=discriminator_width)
```

The generator projects its input to $32 \times 7 \times 7$ feature maps, then doubles the spatial size twice with transposed convolutions. The discriminator halves the image size twice before its final linear map.

Exercise

Trace both networks’ shapes. Calculate the parameters in the projection and each convolution. Which part of this small generator has most of its parameters? Check the total against the executable file.

3 Two optimisation steps

Write $s_r = D(x)$ and $s_f = D(G(z))$ for discriminator logits. The non-saturating losses are

$$L_D = \mathbb{E}[\text{softplus}(-s_r)] + \mathbb{E}[\text{softplus}(s_f)], \quad L_G = \mathbb{E}[\text{softplus}(-s_f)].$$

The discriminator should assign large logits to real images and small logits to generated images. The generator tries to increase the discriminator's logit on its outputs. We perform one update of each network per iteration.

```
def losses(real_logits, fake_logits):  
    loss_d = F.softplus(-real_logits).mean() + F.softplus(fake_logits).mean()  
    loss_g = F.softplus(-fake_logits).mean()  
    return loss_d, loss_g  
print(losses(torch.zeros(8), torch.zeros(8)))
```

When updating D , use `fake.detach()` so that this backward pass does not update G . After the discriminator update, evaluate its current parameters on the generator output for the generator loss. Freeze discriminator parameters for this second backward pass, while retaining the derivative with respect to its input.

Exercise

Find these operations in the training loop below. Explain why wrapping the discriminator's generator-step forward pass in `no_grad()` would prevent the generator learning. What values do the two losses have when all logits are zero?

4 Inspect the generated batches

The MNIST model above has 45,202 parameters and trains for 2,000 updates with batch size 64 and learning rate 0.001.

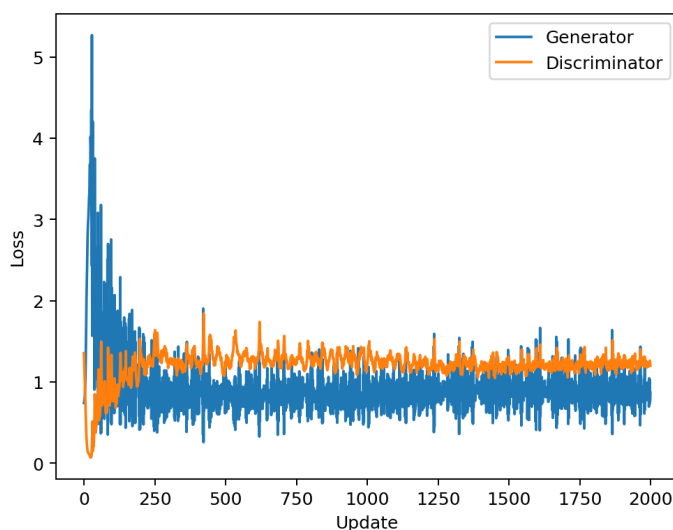


Figure 1: A batch of 64 generated digits, alongside example generator and discriminator loss curves from a separate training run. Several digits are clear, while others have broken or ambiguous strokes.

Examine the generated images for background, stroke width, loops, repeated shapes and missing digit classes. Interpret these alongside both losses, bearing in mind that each network changes the objective faced by the other, so their losses can move in different directions.

Exercise

Describe two features that the small generator has learned and two that it has not. Compare these observations with the loss traces. Why would a low generator loss alone be insufficient evidence of good image generation?

5 Compare image distributions

Compare whole grids of real and generated digits, looking at the range of digit classes, stroke widths and slants. We can also compare image distributions through features from a fixed neural network: similar images should have similar feature vectors. A repeated image has no variation in its features, even if it is individually convincing. The CIFAR-10 challenge below uses KID to compare these feature distributions.

Exercise

Consider a generator that produces the same clear digit on every call. What would you expect to see in the generated feature covariance? How could a generator obtain a good average feature vector while producing poor samples?

6 Improve the MNIST model

Begin by increasing the channel counts modestly, then investigate normalisation of the projected 7×7 features, discriminator width or learning rate. When comparing these changes, count the parameters of *both* networks and retain one discriminator update and one generator update per iteration, with no additional discriminator batches.

Compare your new sample batches with the toy run. Keep the same seed and compare complete grids. The following cell trains the displayed model and then interpolates between two latent vectors, generating images along $(1 - t)z_a + tz_b$:

```
from torchvision.datasets import MNIST
from torchvision.utils import make_grid
import matplotlib.pyplot as plt
```

```
def data():
    dataset = MNIST("data", train=True, download=True)
    order = torch.randperm(60000, generator=torch.Generator().manual_seed(3801))
    return dataset.data[order[:50000]].unsqueeze(1).float()/127.5 - 1
```

```
def train(model, images, steps=2000, batch=64, learning_rate=.001):
    device = next(model.parameters()).device
    generator, discriminator = model.generator, model.discriminator
    opt_g = torch.optim.Adam(generator.parameters(), lr=learning_rate, betas=(.5, .999))
    opt_d = torch.optim.Adam(discriminator.parameters(), lr=learning_rate, betas=(.5, .999))
    history = []
    model.train()
    for step in range(steps):
        real = images[torch.randint(len(images), (batch,))].to(device)
        fake = generator(torch.randn(batch, generator.latent, device=device))
        loss_d = F.softplus(-discriminator(real)).mean()
        loss_d = loss_d + F.softplus(discriminator(fake.detach())).mean()
        opt_d.zero_grad()
        loss_d.backward()
        opt_d.step()

        discriminator.requires_grad_(False)
        loss_g = F.softplus(-discriminator(fake)).mean()
        opt_g.zero_grad()
        loss_g.backward()
        opt_g.step()
        discriminator.requires_grad_(True)
        history.append([loss_g.item(), loss_d.item()])
    return history
```

```

torch.manual_seed(3801)
model = Model().cuda()
history = train(model, data())
plt.plot(history)
plt.legend(["Generator", "Discriminator"])
plt.xlabel("Update"); plt.ylabel("Loss"); plt.show()
model.eval()
torch.manual_seed(921)
with torch.no_grad():
    samples = model.generator(torch.randn(64, model.generator.latent, device="cuda"))
plt.imshow(make_grid((samples+1)/2, nrow=8).permute(1, 2, 0).cpu())
plt.axis("off"); plt.show()

```

Now interpolate between two latent vectors:

```

a, b = torch.randn(2, model.generator.latent, device="cuda")
t = torch.linspace(0, 1, 9, device="cuda")[:, None]
with torch.no_grad():
    digits = model.generator((1-t)*a+t*b)
plt.imshow(make_grid((digits+1)/2, nrow=9).permute(1, 2, 0).cpu())
plt.axis("off"); plt.show()

```



Figure 2: Nine points along a straight line between two latent vectors, using the trained MNIST generator.

Which strokes appear or disappear gradually? Repeat with your wider MNIST generator.

7 CIFAR-10 challenge: generate horses

For the submission, we will use a fixed split of the CIFAR-10 horse class: 4,000 training photographs and 1,000 validation photographs. Every image has shape $3 \times 32 \times 32$, with pixel values scaled to $[-1, 1]$. The submission runner downloads this dataset and the fixed evaluation weights when first needed. Run `dl-practical5.py`, either as a complete notebook cell or from the terminal:

```
python dl-practical5.py
```

Display the first training batch:

```

from torchvision.datasets import CIFAR10
source = CIFAR10("data", train=True, download=True)
indices = torch.where(torch.tensor(source.targets) == 7)[0]
order = torch.randperm(5000, generator=torch.Generator().manual_seed(3802))
indices = indices[order[:4000]]
real = torch.tensor(source.data[indices]).permute(0, 3, 1, 2).float()/127.5 - 1
plt.imshow(make_grid((real[:64]+1)/2, nrow=8).permute(1, 2, 0))
plt.axis("off"); plt.show()

```

The CIFAR-10 starter follows the MNIST model above. Change the initial 7×7 feature maps to 8×8 , and use three colour channels at the generator output and discriminator input. The generator still has two transposed convolutions, producing 16×16 then 32×32 images. The discriminator has two convolutions and a final linear map. The starter uses the default PyTorch initialisation and one batch-normalisation layer in the generator. It has 54,868 parameters and begins with 2,000 iterations, each updating both networks once, at learning rate 0.001. These outputs were generated using seed 3801 and the settings shown below, with the score measured on the complete fixed validation protocol. Small numerical differences between environments are expected.

dcgan: 54,868 parameters, 2,000 iterations, batch 64; KID x 1000: 263.222.

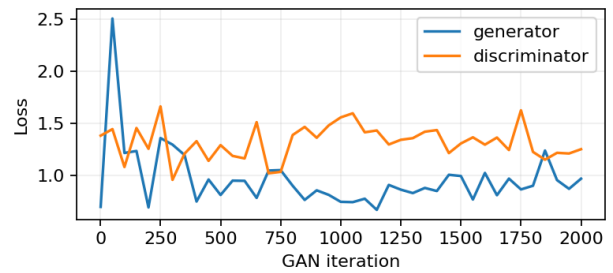


Figure 3: First fixed sample batch and training losses from this run. Running the program creates all sample files.

Exercise

1. Trace the spatial sizes and channel counts in the RGB file. Identify the changes from the MNIST model needed to produce 32×32 RGB images.
2. Inspect the real training images displayed above. Compare the baseline's colours, backgrounds, legs and body shapes with those images.
3. Increase generator and discriminator capacity separately. Keep the learning rate and update count fixed for the first comparison. Which change improves the samples most?
4. Replace an upsampling convolution with nearest-neighbour upsampling followed by a 3×3 convolution. Compare repeated grid patterns and sharpness.
5. Adjust learning rate and training length within the limits below. Compare both loss traces, all sample grids and the evaluation score for each run.

What changes can achieve

Here is an example produced within the assignment constraints, with $\text{KID} \times 1000 = 28.820$.

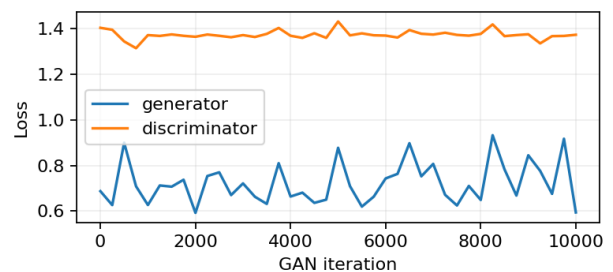


Figure 4: First fixed sample batch and training losses from this run.

8 Score the CIFAR-10 samples with KID

Install `torch-fidelity` in the PyTorch environment. In a notebook, run `%pip install torch-fidelity` in a separate cell before the program. The template calculates Kernel Inception Distance (KID) automatically, using 1,000 generated images and the 1,000 validation images.

KID compares the two image distributions through features from a fixed Inception network; lower scores indicate a closer match. The printed score is KID multiplied by 1,000, so a displayed value of 80 appears as `"kid": 0.080` in the JSON. All 1,000 generated images are saved in ten PNG grids. Compare their variety and visual quality alongside the score, and keep the supplied sampling and evaluation code unchanged.

Further reading. Radford et al., *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*; Bińkowski et al., *Demystifying MMD GANs*, Section 4 (KID).

For further reading, see MobileNetV5 (efficient architectures) and spectral normalisation (GAN training).

9 Submission

Download `d1-practical5.py` , set `STUDENT_ID` and edit the model and training settings. Run the complete file or paste the whole program into one notebook cell. It trains, scores and creates the ZIP below; upload this to the corresponding Ultra item.

```
lab-05-studentID.zip
lab-05-code.py
lab-05-results.json
samples/
  batch-01.png
  batch-02.png
  ...
  batch-10.png
```

Details and constraints

- Generate ten batches of 100 model samples, arranged as 10 by 10 grids. Each PNG is 320 by 320 RGB 8-bit pixels, with no padding or resizing. These 1,000 images are also used for KID.
- Use at most 1,000,000 parameters and buffer elements across both networks, 10,000 generator updates and 10,000 discriminator updates, with a batch size of at most 64. Use one update of each network per iteration.
- Train from fresh weights on the fixed 4,000 CIFAR-10 horse training images at 32 by 32 RGB. Reserve the Inception network for evaluation.
- Use no extra data, pretrained models, distillation teacher, gradient accumulation or extra training batches per update.
- Keep the data split, loss and sampling procedure fixed. Evaluate KID against the 1,000 validation images; JSON stores raw KID and the printed score is KID x 1,000.
- Put a brief abstract at the top of the Python file, with references and AI acknowledgements in comments. The program writes the JSON and sample files; submit these unchanged.

Marking

- Marks reflect the quality and simplicity of your solution and its measured performance within the constraints. Aim for a KID x 1,000 below 263.2.
- A pass requires a small improvement over the baseline. A 2:1 requires significant improvements using established techniques discussed in the course. A high first requires substantial improvements through well-implemented state-of-the-art methods, for example by integrating methods from recent papers, producing high-quality samples and scores within the budget.
- The exact marking function is withheld to avoid gamification.

Solutions for this assignment cannot later be publicised without request. Requests will be considered on a case-by-case basis after marking.